

Capstone Project Prospectus

Project Title: Quantifying the Operational ROI and Engineering Bottlenecks of Internal Developer Platforms (IDP) Using Predictive Telemetry

1. General Idea / Overview

Modern enterprise engineering organizations heavily invest in Internal Developer Platforms (IDPs) to abstract infrastructure complexities (such as Kubernetes orchestration) and accelerate software delivery. However, technology executives lack empirical, data-driven frameworks to measure platform ROI, optimize developer velocity, and pinpoint remaining software development lifecycle (SDLC) bottlenecks.

This project addresses the central question: **Can we leverage engineering infrastructure telemetry to predict software deployment cycle times, isolate systemic technical debt, and mathematically segment developer squads to strategically optimize platform adoption?**

2. Data Strategy & Feature Matrix

The project will utilize an engineered telemetry dataset simulating a mature enterprise SDLC environment (~1,000 unique deployment lifecycles). The data schema maps directly to operational realities and is structured as follows:

Target Variable (Dependent Variable)

- Deployment_Cycle_Time (Continuous / Hours):** The absolute elapsed time from a developer opening a Pull Request (PR) to that service running successfully in a production-ready Kubernetes namespace.

Independent Features Matrix (X)

Category	Feature Name	Data Type	Description
Platform Adoption	IDP_API_Calls	Continuous	Count of developer interactions with the platform API per sprint.
	GitOps_Auto_Sync	Binary (0/1)	Whether automated GitOps deployment pipelines are enabled.
	Manual_Kubectl_Cou	Continuous	Count of direct

	nt		cluster bypass commands executed by developers.
Architecture & Debt	Microservice_Deps	Discrete	Total number of downstream architectural dependencies.
	Helm_Template_Lines	Continuous	Size and complexity of underlying infrastructure-as-code manifests.
	Repo_Code_Churn	Continuous	Total lines of code modified/deleted within the deployment window.
Team Demographics	Team_Size	Discrete	Total number of active contributors on the engineering squad.
	Offshore_Ratio	Continuous	Percentage of the team operating across distributed time zones.

3. Methodological Framework & Techniques

To solve this multi-dimensional problem, the project implements three core algorithms from the Module 11 curriculum sequentially to extract high-leverage insights:

Technique 1: Principal Component Analysis (PCA)

- **Application:** Infrastructure telemetry is inherently collinear (e.g., highly complex microservices naturally generate larger Helm templates and higher code churn). PCA will be used to reduce this

high-dimensional noise, collapsing correlated metrics into 2 or 3 principal components that capture maximum variance representing "Systemic Structural Complexity."

- **Strategic Objective:** Cleanly isolates pure architectural complexity from behavioral adoption patterns before entering predictive models.

Technique 2: Lasso (L_1) and Ridge (L_2) Regularized Regression

- **Application:** A regularized linear model will be trained to predict the continuous target `Deployment_Cycle_Time`. Lasso regression will be specifically leveraged to apply an L_1 penalty, forcing non-essential or weak feature coefficients to absolute zero.
- **Strategic Objective:** Acts as an objective, mathematical feature selector. The non-zero coefficients surviving the Lasso penalty will expose the absolute strongest macro drivers of deployment friction or acceleration for executive review.

Technique 3: K-Means Clustering

- **Application:** Unsupervised K-Means clustering will be executed on the engineering squads utilizing feature dimensions of platform adoption and complexity. The Elbow Method and Silhouette Analysis will determine the mathematically optimal number of cohorts (\$K\$).
- **Strategic Objective:** Instead of issuing broad, ineffective company-wide mandates, leadership can use these empirical profiles (e.g., "High-Velocity Platform Adopters," "Legacy Scripting Holdouts") to deploy highly targeted training and infrastructure support.

4. Strategic & Operational Impact (The "So What?")

Upon successful completion, this project transitions from an algorithmic workbook into an executive roadmap:

- **Justifies Tech Spend:** Provides quantitative proof of how much platform adoption actively decreases time-to-market.
- **Optimizes Resource Allocation:** Flags exact architectural patterns (e.g., excessive downstream microservice dependencies) that drag down velocity, shifting engineering focus from guessing bottlenecks to fixing them.
- **Enforces Corporate Governance:** Empirically proves the operational cost of bypassing standard platform guardrails (manual overrides), paving the way for data-backed compliance policies.